COURSE DELIVERABLE

SE-4011 Software Measurement and Metrics

FAST Societies Management System

Consolidated technical report — software metrics, reliability, usability, and estimation analysis

SUBMITTED TO

Dr. Atif Jillani

Section **SE-D** · Department of Software Engineering

GROUP MEMBERS

Name	Roll number
Hammad Zahid	22I-2433
Abdullah Asif	22I-1527
Dawood Qammar	22I-2522

Source repository: https://github.com/Ninjaa-aa/fast-sms

Table of Contents

1. Project Overview

Our university campus has multiple student societies such as Gaming Society, Sports Society, Developers Club, Literary Society, and Media Society. These societies organize events, workshops, competitions, recruitment drives, and awareness campaigns throughout the semester. The absence of a centralized digital platform for managing student societies creates communication gaps, inefficient event handling, poor record management, and lack of administrative oversight.

1.1 Functional Requirements

1.1.1 Student Functional Requirements

1. Students shall be able to create accounts and log in securely.
2. Students shall be able to browse available societies.
3. Students shall be able to apply for membership in societies.
4. Students shall be able to join multiple societies.
5. Students shall be able to view upcoming society events.
6. Students shall be able to register for events online.
7. Students shall be able to view their membership status.
8. Students shall be able to view event tickets/passes.

1.1.2 Society Functional Requirements

1. Society heads shall be able to create and manage society profiles.
2. Society heads shall be able to approve or reject membership requests.
3. Society members shall be able to manage internal member lists.
4. Societies shall be able to create, update, and cancel events.
5. Societies shall be able to assign tasks to members.
6. Societies shall be able to generate reports of members and events.

1.1.3 Admin Functional Requirements

1. Admin shall be able to manage all student accounts.
2. Admin shall be able to create, approve, suspend, or delete societies.
3. Admin shall be able to monitor all society activities.
4. Admin shall be able to approve event requests.
5. Admin shall be able to generate university-wide reports.

1.2 System Architecture

The FAST Societies Management System follows a layered architecture with clear separation between the presentation layer (Windows Forms), the data access layer (DAL), and the database (SQL Server). Helper and configuration modules support cross-cutting concerns such as session management, password hashing, and environment-based configuration.

1.3 Database Schema (Task 1)

The database consists of seven relational tables designed to support the core operations of the societies management system. Below is the schema definition followed by a summary table.

```
CREATE TABLE Users (
    UserID        INT IDENTITY(1,1) PRIMARY KEY,
    FullName      NVARCHAR(100)  NOT NULL,
    Email         NVARCHAR(100)  NOT NULL UNIQUE,
    PasswordHash  NVARCHAR(255)  NOT NULL,
    Role          NVARCHAR(20)   NOT NULL
        CHECK (Role IN ('Student','SocietyHead','Admin')),
    CreatedAt     DATETIME DEFAULT GETDATE()
);

CREATE TABLE Societies (
    SocietyID    INT IDENTITY(1,1) PRIMARY KEY,
    Name         NVARCHAR(100)  NOT NULL,
    Description  NVARCHAR(500),
    HeadUserID   INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
    Status       NVARCHAR(20) DEFAULT 'Pending'
        CHECK (Status IN ('Pending','Active','Suspended')),
    CreatedAt    DATETIME DEFAULT GETDATE()
);

CREATE TABLE Memberships (
    MembershipID INT IDENTITY(1,1) PRIMARY KEY,
    UserID       INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
    SocietyID    INT NOT NULL FOREIGN KEY REFERENCES Societies(SocietyID),
    Status       NVARCHAR(20) DEFAULT 'Pending'
        CHECK (Status IN ('Pending','Approved','Rejected')),
    AppliedAt    DATETIME DEFAULT GETDATE()
);

CREATE TABLE Events (
    EventID    INT IDENTITY(1,1) PRIMARY KEY,
    SocietyID  INT NOT NULL FOREIGN KEY REFERENCES Societies(SocietyID),
    Title      NVARCHAR(200)  NOT NULL,
    Description NVARCHAR(1000),
    EventDate  DATETIME       NOT NULL,
    Venue      NVARCHAR(200),
    Status     NVARCHAR(20)  DEFAULT 'Pending'
        CHECK (Status IN ('Pending','Approved','Cancelled')),
    CreatedAt  DATETIME DEFAULT GETDATE()
);

CREATE TABLE EventRegistrations (
    RegistrationID INT IDENTITY(1,1) PRIMARY KEY,
    EventID        INT NOT NULL FOREIGN KEY REFERENCES Events(EventID),
    UserID         INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
    TicketCode     NVARCHAR(50) NOT NULL,
    RegisteredAt   DATETIME DEFAULT GETDATE()
```

```
);

CREATE TABLE Tasks (
    TaskID      INT IDENTITY(1,1) PRIMARY KEY,
    SocietyID   INT NOT NULL FOREIGN KEY REFERENCES Societies(SocietyID),
    Title       NVARCHAR(200)  NOT NULL,
    Description NVARCHAR(500),
    AssignedTo  INT FOREIGN KEY REFERENCES Users(UserID),
    AssignedBy  INT FOREIGN KEY REFERENCES Users(UserID),
    DueDate     DATETIME,
    Status      NVARCHAR(20) DEFAULT 'Pending'
        CHECK (Status IN ('Pending','Completed')),
    CreatedAt   DATETIME DEFAULT GETDATE()
);

CREATE TABLE Announcements (
    AnnouncementID INT IDENTITY(1,1) PRIMARY KEY,
    SocietyID      INT NOT NULL FOREIGN KEY REFERENCES Societies(SocietyID),
    Title          NVARCHAR(200) NOT NULL,
    Content        NVARCHAR(2000),
    CreatedAt      DATETIME DEFAULT GETDATE()
);
```

Table	Primary Key	Key Columns	Relationships
Users	UserID	FullName, Email, PasswordHash, Role	—
Societies	SocietyID	Name, Description, Status	FK: HeadUserID → Users
Memberships	MembershipID	Status, AppliedAt	FK: UserID → Users, SocietyID → Societies
Events	EventID	Title, EventDate, Venue, Status	FK: SocietyID → Societies
EventRegistrations	RegistrationID	TicketCode	FK: EventID → Events, UserID → Users
Tasks	TaskID	Title, DueDate, Status	FK: SocietyID → Societies, AssignedTo/By → Users
Announcements	AnnouncementID	Title, Content	FK: SocietyID → Societies

2. Task 2 — Cyclomatic Complexity & Test Cases

Cyclomatic Complexity (CC) measures the number of linearly independent paths through a method. It is computed as **CC = 1 + number of decision points**, where decision points include *if*, *else if*, *while*, *for*, *foreach*, *switch-case*, *catch*, ||, *&&*, and *?:* operators. A higher CC indicates more complex logic and more test cases required for full path coverage.

2.1 EnvConfig

#	Method	Decision Points	CC	Test Cases
1	Load()	if (1)	2	TC1: CONNECTION_STRING set. TC2: Empty, builds from components.
2	EnsureDatabaseInConnectionString()	|| (1), if (1), ?: (1)	4	TC1: Has Initial Catalog. TC2: Has Database=. TC3: Missing, ends with ;. TC4: Missing, no ;.

2.2 DBHelper

#	Method	Decision Points	CC	Test Cases
3	GetConnection()	0	1	TC1: Valid conn string returns open connection.
4	ExecuteNonQuery()	0	1	TC1: Valid INSERT returns rows > 0.
5	ExecuteReader()	0	1	TC1: Valid SELECT returns DataTable.
6	ExecuteScalar()	0	1	TC1: Valid COUNT returns scalar.

2.3 UserDAL

#	Method	Decision Points	CC	Test Cases
7	GetByEmail()	if (1)	2	TC1: Email exists, returns User. TC2: Not found, returns null.
8	EmailExists()	0	1	TC1: Email exists, returns true.
9	Register()	0	1	TC1: Valid User inserts, returns true.
10	GetAll()	0	1	TC1: Returns DataTable with rows.
11	Search()	0	1	TC1: Search term matches rows.
12	Delete()	0	1	TC1: Valid userId deletes row.
13	GetCount()	0	1	TC1: Returns integer count.
14	GetSocietyHeads()	0	1	TC1: Returns SocietyHead users.
15	MapUser()	0	1	TC1: Valid DataRow maps to User.

2.4 SocietyDAL

#	Method	Decision Points	CC	Test Cases
16	GetActive()	0	1	TC1: Active societies returned.
17	GetAll()	0	1	TC1: All societies with head names.
18	GetByHead()	0	1	TC1: Returns society for head user.
19	GetActiveCount()	0	1	TC1: Returns active count.
20	Create()	0	1	TC1: Inserts and returns true.
21	Approve()	0	1	TC1: Sets status to Active.
22	Suspend()	0	1	TC1: Sets status to Suspended.
23	Delete()	0	1	TC1: Deletes society row.
24	GetPerformanceSummary()	0	1	TC1: Returns aggregated stats.

2.5 MembershipDAL

#	Method	Decision Points	CC	Test Cases
25	HasApplied()	0	1	TC1: Has membership, returns true.
26	Apply()	0	1	TC1: Inserts pending row.
27	GetByUser()	0	1	TC1: Returns joined DataTable.
28	GetBySociety()	0	1	TC1: Returns society members.
29	GetBySocietyFiltered()	0	1	TC1: Returns filtered rows.
30	Approve()	0	1	TC1: Sets status to Approved.
31	Reject()	0	1	TC1: Sets status to Rejected.
32	GetApprovedMembers()	0	1	TC1: Returns approved list.
33	GetAll()	0	1	TC1: Returns full joined table.

2.6 EventDAL

#	Method	Decision Points	CC	Test Cases
34	GetUpcomingApproved()	0	1	TC1: Future approved events.
35	IsRegistered()	0	1	TC1: User registered, returns true.
36	Register()	0	1	TC1: Inserts with ticket code.
37	GetTicketsByUser()	0	1	TC1: Returns user tickets.
38	GetBySociety()	0	1	TC1: Returns society events.
39	Create()	0	1	TC1: Inserts Pending event.
40	Cancel()	0	1	TC1: Sets status to Cancelled.
41	GetPending()	0	1	TC1: Returns pending events.
42	Approve()	0	1	TC1: Sets status to Approved.
43	Reject()	0	1	TC1: Sets status to Cancelled.
44	GetPendingCount()	0	1	TC1: Returns pending count.
45	GetAll()	0	1	TC1: Returns full event table.

2.7 TaskDAL

#	Method	Decision Points	CC	Test Cases
46	GetBySociety()	0	1	TC1: Returns society tasks.
47	Create()	0	1	TC1: Inserts Pending task.
48	MarkComplete()	0	1	TC1: Sets status to Completed.

2.8 AnnouncementDAL

#	Method	Decision Points	CC	Test Cases
49	GetBySociety()	0	1	TC1: Returns announcements.
50	Create()	0	1	TC1: Inserts announcement.

2.9 Session

#	Method	Decision Points	CC	Test Cases
51	Clear()	0	1	TC1: All properties reset.

2.10 PasswordHasher

#	Method	Decision Points	CC	Test Cases
52	Hash()	0	1	TC1: Returns BCrypt hash string.

#	Method	Decision Points	CC	Test Cases
53	Verify()	0	1	TC1: Correct password returns true.

2.11 Program

#	Method	Decision Points	CC	Test Cases
54	Main()	0	1	TC1: App starts, LoginForm displayed.

2.12 LoginForm

#	Method	Decision Points	CC	Test Cases
55	LoginForm() ctor	0	1	TC1: Form initialises.
56	BtnLogin_Click()	if+|| (2), if+|| (2), switch 3 (3), catch (1)	9	TC1-2: Empty fields. TC3-4: Invalid credentials. TC5-7: Role dispatch. TC8: Unknown role. TC9: DB error.
57	LnkRegister_LinkClicked()	0	1	TC1: Shows RegisterForm.

2.13 RegisterForm

#	Method	Decision Points	CC	Test Cases
58	RegisterForm() ctor	0	1	TC1: Form initialises.
59	BtnRegister_Click()	if+||+|| (3), if (1), catch (1), if (1), if (1)	8	TC1-3: Empty fields. TC4: Password mismatch. TC5: Email exists. TC6: Success. TC7: DB returns false. TC8: DB error.
60	LnkLogin_LinkClicked()	0	1	TC1: Navigates to login.
61	NavigateToLogin()	0	1	TC1: Creates LoginForm.

2.14 StudentDashboard

#	Method	Decision Points	CC	Test Cases
62	StudentDashboard() ctor	0	1	TC1: Welcome label set.
63	BtnBrowseSocieties_Click()	0	1	TC1: Shows BrowseSocieties.
64	BtnMyMemberships_Click()	0	1	TC1: Shows MyMemberships.
65	BtnBrowseEvents_Click()	0	1	TC1: Shows BrowseEvents.
66	BtnMyTickets_Click()	0	1	TC1: Shows MyTickets.
67	BtnLogout_Click()	0	1	TC1: Session cleared, logout.

2.15 BrowseSocieties

#	Method	Decision Points	CC	Test Cases
68	BrowseSocieties() ctor	0	1	TC1: Form initialises.
69	BrowseSocieties_Load()	0	1	TC1: Calls LoadSocieties.
70	LoadSocieties()	catch (1), if (1)	3	TC1: Loaded, col hidden. TC2: No SocietyID col. TC3: DB error.
71	BtnApply_Click()	if (1), catch (1), if (1), if (1)	5	TC1: No selection. TC2: Already applied. TC3: Success. TC4: Fails. TC5: DB error.
72	BtnBack_Click()	0	1	TC1: Returns to dashboard.

2.16 MyMemberships

#	Method	Decision Points	CC	Test Cases
73	MyMemberships() ctor	0	1	TC1: Form initialises.
74	MyMemberships_Load()	catch (1)	2	TC1: Grid loads. TC2: DB error.
75	BtnBack_Click()	0	1	TC1: Returns to dashboard.

2.17 BrowseEvents

#	Method	Decision Points	CC	Test Cases
76	BrowseEvents() ctor	0	1	TC1: Form initialises.
77	BrowseEvents_Load()	0	1	TC1: Calls LoadEvents.
78	LoadEvents()	catch (1), if (1)	3	TC1: Loaded, col hidden. TC2: No EventID col. TC3: DB error.
79	BtnRegister_Click()	if (1), catch (1), if (1), if (1)	5	TC1: No selection. TC2: Already registered. TC3: Success. TC4: Fails. TC5: DB error.
80	BtnBack_Click()	0	1	TC1: Returns to dashboard.

2.18 MyTickets

#	Method	Decision Points	CC	Test Cases
81	MyTickets() ctor	0	1	TC1: Form initialises.
82	MyTickets_Load()	catch (1)	2	TC1: Grid loads. TC2: DB error.
83	BtnBack_Click()	0	1	TC1: Returns to dashboard.

2.19 SocietyDashboard

#	Method	Decision Points	CC	Test Cases
84	SocietyDashboard() ctor	0	1	TC1: Form initialises.
85	SocietyDashboard_Load()	catch (1), foreach (1), if (1), if (1)	5	TC1: Active society found. TC2: Pending only. TC3: Multiple, finds active. TC4: No societies. TC5: DB error.
86	EnableNavButtons()	0	1	TC1: Buttons enabled/disabled.
87	BtnManageMembers_Click()	0	1	TC1: Shows ManageMembers.
88	BtnManageEvents_Click()	0	1	TC1: Shows ManageEvents.
89	BtnManageTasks_Click()	0	1	TC1: Shows ManageTasks.
90	BtnReports_Click()	0	1	TC1: Shows SocietyReports.
91	BtnLogout_Click()	0	1	TC1: Session cleared.

2.20 ManageMembers

#	Method	Decision Points	CC	Test Cases
92	ManageMembers() ctor	0	1	TC1: Form initialises.
93	ManageMembers_Load()	0	1	TC1: Calls LoadMembers.
94	LoadMembers()	catch (1), if (1), ?: (1), if (1)	5	TC1: SocietyID null. TC2: Filter All. TC3: Filter Pending. TC4: Col hidden. TC5: DB error.
95	CmbFilter_SelectedIndexChanged()	0	1	TC1: Calls LoadMembers.
96	BtnApprove_Click()	if (1), catch (1)	3	TC1: No selection. TC2: Approved. TC3: DB error.
97	BtnReject_Click()	if (1), catch (1)	3	TC1: No selection. TC2: Rejected. TC3: DB error.
98	BtnBack_Click()	0	1	TC1: Returns to dashboard.

2.21 ManageEvents

#	Method	Decision Points	CC	Test Cases
99	ManageEvents() ctor	0	1	TC1: Form initialises.
100	ManageEvents_Load()	0	1	TC1: Calls LoadEvents.
101	LoadEvents()	catch (1), if (1), if (1)	4	TC1: SocietyID null. TC2: Col hidden. TC3: No col. TC4: DB error.
102	BtnCreate_Click()	if (1), catch (1)	3	TC1: Dialog cancelled. TC2: Created. TC3: DB error.
103	BtnCancelEvent_Click()	if (1), if (1), catch (1)	4	TC1: No selection. TC2: User says No. TC3: Cancelled. TC4: DB error.
104	BtnBack_Click()	0	1	TC1: Returns to dashboard.

2.22 CreateEventDialog

#	Method	Decision Points	CC	Test Cases
105	CreateEventDialog() ctor	if+|| (2)	3	TC1: Valid input. TC2: Empty title. TC3: Empty venue.

2.23 ManageTasks

#	Method	Decision Points	CC	Test Cases
106	ManageTasks() ctor	0	1	TC1: Form initialises.
107	ManageTasks_Load()	0	1	TC1: Calls LoadTasks.
108	LoadTasks()	catch (1), if (1), if (1)	4	TC1: SocietyID null. TC2: Col hidden. TC3: No col. TC4: DB error.
109	BtnAssign_Click()	if (1), if (1), catch (1)	4	TC1: SocietyID null. TC2: Dialog cancelled. TC3: Created. TC4: DB error.
110	BtnComplete_Click()	if (1), catch (1)	3	TC1: No selection. TC2: Completed. TC3: DB error.
111	BtnBack_Click()	0	1	TC1: Returns to dashboard.

2.24 AssignTaskDialog

#	Method	Decision Points	CC	Test Cases
112	AssignTaskDialog() ctor	if+|| (2)	3	TC1: Valid input. TC2: No member. TC3: Empty title.

2.25 SocietyReports

#	Method	Decision Points	CC	Test Cases
113	SocietyReports() ctor	0	1	TC1: Form initialises.
114	SocietyReports_Load()	0	1	TC1: Calls LoadReports.
115	LoadReports()	catch (1), if (1), if (1), if (1)	5	TC1: SocietyID null. TC2: Both cols hidden. TC3: No UserID col. TC4: No EventID col. TC5: DB error.
116	BtnRefresh_Click()	0	1	TC1: Calls LoadReports.
117	BtnBack_Click()	0	1	TC1: Returns to dashboard.

2.26 AdminDashboard

#	Method	Decision Points	CC	Test Cases
118	AdminDashboard() ctor	0	1	TC1: Form initialises.
119	AdminDashboard_Load()	catch (1)	2	TC1: Stats loaded. TC2: DB error.
120	BtnManageUsers_Click()	0	1	TC1: Shows ManageUsers.
121	BtnManageSocieties_Click()	0	1	TC1: Shows ManageSocieties.
122	BtnApproveEvents_Click()	0	1	TC1: Shows ApproveEvents.
123	BtnReports_Click()	0	1	TC1: Shows AdminReports.
124	BtnLogout_Click()	0	1	TC1: Session cleared.

2.27 ManageUsers

#	Method	Decision Points	CC	Test Cases
125	ManageUsers() ctor	0	1	TC1: Form initialises.
126	ManageUsers_Load()	0	1	TC1: Calls LoadUsers.
127	LoadUsers()	catch (1), if (1)	3	TC1: Col hidden. TC2: No col. TC3: DB error.
128	BtnSearch_Click()	catch (1), ?: (1), if (1)	4	TC1: Empty search. TC2: With term. TC3: Col hidden. TC4: DB error.
129	BtnDelete_Click()	if (1), if (1), catch (1)	4	TC1: No selection. TC2: User says No. TC3: Deleted. TC4: DB error.
130	BtnBack_Click()	0	1	TC1: Returns to dashboard.

2.28 ManageSocieties

#	Method	Decision Points	CC	Test Cases
131	ManageSocieties() ctor	0	1	TC1: Form initialises.
132	ManageSocieties_Load()	0	1	TC1: Calls LoadSocieties.
133	LoadSocieties()	catch (1), if (1)	3	TC1: Col hidden. TC2: No col. TC3: DB error.
134	BtnCreate_Click()	if (1), catch (1)	3	TC1: Cancelled. TC2: Created. TC3: DB error.
135	BtnApprove_Click()	if (1), catch (1)	3	TC1: No selection. TC2: Approved. TC3: DB error.
136	BtnSuspend_Click()	if (1), catch (1)	3	TC1: No selection. TC2: Suspended. TC3: DB error.
137	BtnDelete_Click()	if (1), if (1), catch (1)	4	TC1: No selection. TC2: User says No. TC3: Deleted. TC4: DB error.
138	BtnBack_Click()	0	1	TC1: Returns to dashboard.

2.29 CreateSocietyDialog

#	Method	Decision Points	CC	Test Cases
139	CreateSocietyDialog() ctor	if+|| (2)	3	TC1: Valid input. TC2: Empty name. TC3: No head.

2.30 ApproveEvents

#	Method	Decision Points	CC	Test Cases
140	ApproveEvents() ctor	0	1	TC1: Form initialises.
141	ApproveEvents_Load()	0	1	TC1: Calls LoadPendingEvents.
142	LoadPendingEvents()	catch (1), if (1)	3	TC1: Col hidden. TC2: No col. TC3: DB error.
143	BtnApprove_Click()	if (1), catch (1)	3	TC1: No selection. TC2: Approved. TC3: DB error.
144	BtnReject_Click()	if (1), catch (1)	3	TC1: No selection. TC2: Rejected. TC3: DB error.
145	BtnBack_Click()	0	1	TC1: Returns to dashboard.

2.31 AdminReports

#	Method	Decision Points	CC	Test Cases
146	AdminReports() ctor	0	1	TC1: Form initialises.
147	AdminReports_Load()	0	1	TC1: Calls LoadReports.
148	LoadReports()	catch (1)	2	TC1: Reports loaded. TC2: DB error.
149	BtnRefresh_Click()	0	1	TC1: Calls LoadReports.
150	BtnBack_Click()	0	1	TC1: Returns to dashboard.

2.32 Summary Statistics

Metric	Value
Total Methods Analysed	150
Total CC (sum)	249
Average CC	1.66
Median CC	1
Highest CC	9 — LoginForm.BtnLogin_Click
Second Highest CC	8 — RegisterForm.BtnRegister_Click
Lowest CC	1 — 105 methods (70.0%)

2.33 CC Distribution by Module

Module	Methods	Sum CC	Avg CC
Config (EnvConfig)	2	6	3.00
DAL (7 classes)	50	60	1.20
Helpers	3	3	1.00
Entry (Program)	1	1	1.00
Forms/Auth	7	22	3.14
Forms/Student	16	32	2.00
Forms/Society (incl. dialogs)	34	72	2.12
Forms/Admin (incl. dialog)	31	53	1.71

2.34 Methods Needing Refactoring (CC ≥ 5)

#	Class	Method	CC	Suggestion
1	LoginForm	BtnLogin_Click	9	Extract role-dispatch; split validation.
2	RegisterForm	BtnRegister_Click	8	Extract ValidateInput() method.
3	BrowseSocieties	BtnApply_Click	5	Extract "already applied" check.
4	BrowseEvents	BtnRegister_Click	5	Mirrors BrowseSocieties pattern.
5	SocietyDashboard	SocietyDashboard_Load	5	Extract "find active society" helper.
6	ManageMembers	LoadMembers	5	Filter + null check + column hide.
7	SocietyReports	LoadReports	5	Loads two grids with ID hiding.

Overall, the system exhibits low cyclomatic complexity with an average CC of 1.66 across 150 methods. The majority (70%) of methods have CC = 1, indicating straightforward, single-path logic. Only 7 methods exceed the CC ≥ 5 threshold, and even the highest (CC = 9 in BtnLogin_Click) is well below the critical threshold of 20. The codebase is maintainable and testable, requiring a minimum of 249 test cases for full path coverage.

3. Task 3 — Structural Metric: Best Module Justification

This section evaluates every class in the system against three structural metrics — **Lines of Code (LOC)**, **Fan-Out** (number of distinct external classes/modules referenced), and **Comment Ratio** (comment lines / total lines × 100). The goal is to identify the *best* and *worst* modules based on a composite assessment of size, coupling, and documentation quality.

3.1 Per-Class Structural Metrics

#	Class	Module	LOC	Fan-Out	Comment Lines	Total Lines	Comment Ratio %
1	EnvConfig	Config	41	0	15	60	25.0
2	DBHelper	DAL	44	1	13	61	21.3
3	UserDAL	DAL	91	2	22	123	17.9
4	SocietyDAL	DAL	80	1	22	112	19.6
5	MembershipDAL	DAL	103	1	21	134	15.7
6	EventDAL	DAL	121	1	27	162	16.7
7	TaskDAL	DAL	41	1	9	55	16.4
8	AnnouncementDAL	DAL	29	1	6	38	15.8
9	Session	Helpers	22	0	6	31	19.4
10	PasswordHasher	Helpers	15	0	6	24	25.0
11	Program	Entry	15	2	3	21	14.3
12	User	Models	11	0	2	15	13.3
13	Society	Models	11	0	2	15	13.3
14	Event	Models	13	0	2	17	11.8
15	Membership	Models	11	0	2	15	13.3
16	TaskItem	Models	14	0	3	19	15.8
17	Announcement	Models	10	0	2	14	14.3
18	LoginForm	Forms/Auth	63	7	7	78	9.0
19	RegisterForm	Forms/Auth	80	4	7	95	7.4
20	StudentDashboard	Forms/Student	41	6	4	51	7.8
21	BrowseSocieties	Forms/Student	67	4	7	81	8.6
22	MyMemberships	Forms/Student	29	3	2	35	5.7
23	BrowseEvents	Forms/Student	67	3	4	78	5.1
24	MyTickets	Forms/Student	29	3	2	35	5.7
25	SocietyDashboard	Forms/Society	85	7	5	98	5.1
26	ManageMembers	Forms/Society	80	3	6	94	6.4
27	ManageEvents	Forms/Society	75	4	7	88	8.0
28	CreateEventDialog	Forms/Society	53	0	2	58	3.4
29	ManageTasks	Forms/Society	74	4	6	86	7.0
30	AssignTaskDialog	Forms/Society	57	1	2	63	3.2
31	SocietyReports	Forms/Society	51	4	2	58	3.4
32	AdminDashboard	Forms/Admin	55	9	4	66	6.1
33	ManageUsers	Forms/Admin	75	2	6	88	6.8
34	ManageSocieties	Forms/Admin	96	3	4	108	3.7
35	CreateSocietyDialog	Forms/Admin	52	1	2	57	3.5
36	ApproveEvents	Forms/Admin	68	2	6	81	7.4
37	AdminReports	Forms/Admin	39	4	2	46	4.3

3.2 Module-Level Summary

Module	Classes	Total LOC	Avg LOC	Avg Fan-Out	Avg Comment Ratio %
Config	1	41	41.0	0.0	25.0
DAL	7	509	72.7	1.1	17.6
Helpers	2	37	18.5	0.0	22.2
Entry	1	15	15.0	2.0	14.3
Models	6	70	11.7	0.0	13.6
Forms/Auth	2	143	71.5	5.5	8.2
Forms/Student	5	233	46.6	3.8	6.6
Forms/Society	7	475	67.9	3.3	5.2
Forms/Admin	6	385	64.2	3.5	5.3

3.3 Best Module: PasswordHasher

PasswordHasher is identified as the best module in the system based on the composite structural assessment:

• **LOC = 15** — The smallest non-model class, indicating excellent adherence to the Single Responsibility Principle (SRP). It does exactly one thing: password hashing and verification.

• **Fan-Out = 0** — Zero coupling to other project classes. It depends only on the external BCrypt library, making it fully self-contained, independently testable, and trivially replaceable.

• **Comment Ratio = 25.0%** — The highest comment ratio tied with EnvConfig. One in four lines is a comment, providing excellent documentation for its cryptographic responsibilities.

3.4 Worst Module: AdminDashboard

AdminDashboard is identified as the worst module due to its extremely high coupling:

• **Fan-Out = 9** — The highest fan-out in the entire system. It directly references 9 external classes (UserDAL, SocietyDAL, EventDAL, ManageUsers, ManageSocieties, ApproveEvents, AdminReports, Session, and MessageBox). Any change in these dependencies risks breaking AdminDashboard.

• **Comment Ratio = 6.1%** — Below average documentation for a class that serves as the central admin navigation hub.

• **LOC = 55** — While not excessively large, the combination of high coupling with moderate size indicates a “God Controller” anti-pattern where the dashboard orchestrates too many concerns.

3.5 Conclusion

The structural analysis reveals a well-layered system where backend modules (Config, DAL, Helpers, Models) exhibit low coupling and good documentation, while frontend Form modules show higher fan-out and lower comment ratios — a common pattern in UI-heavy applications. The DAL layer, despite having the most code (509 LOC across 7 classes), maintains low average fan-out (1.1) and strong comment ratios (17.6%), demonstrating disciplined data-access design. Refactoring efforts should prioritize reducing AdminDashboard’s fan-out by introducing a mediator or command pattern for navigation.

4. Task 4 — CK Metrics Suite

The Chidamber & Kemerer (CK) metrics suite provides six object-oriented design indicators: **WMC** (Weighted Methods per Class — sum of cyclomatic complexities), **DIT** (Depth of Inheritance Tree), **NOC** (Number of Children), **CBO** (Coupling Between Objects), **RFC** (Response For a Class — count of methods that can be invoked in response to a message), and **LCOM** (Lack of Cohesion in Methods — number of method pairs that share no instance variables minus pairs that do, floored at 0).

4.1 Complete CK Metrics

#	Class	Module	WMC	DIT	NOC	CBO	RFC	LCOM
1	EnvConfig	Config	6	0	0	2	7	1
2	DBHelper	DAL	4	0	0	7	13	0
3	UserDAL	DAL	10	0	0	7	15	0
4	SocietyDAL	DAL	9	0	0	6	14	0
5	MembershipDAL	DAL	9	0	0	6	14	0
6	EventDAL	DAL	12	0	0	8	19	0
7	TaskDAL	DAL	3	0	0	2	6	0
8	AnnouncementDAL	DAL	2	0	0	1	5	0
9	Session	Helpers	1	0	0	12	1	0
10	PasswordHasher	Helpers	2	0	0	2	4	0
11	Program	Entry	1	0	0	2	5	0
12	User	Models	0	0	0	2	0	0
13	Society	Models	0	0	0	0	0	0
14	Event	Models	0	0	0	0	0	0
15	Membership	Models	0	0	0	0	0	0
16	TaskItem	Models	0	0	0	0	0	0
17	Announcement	Models	0	0	0	0	0	0
18	LoginForm	Forms/Auth	11	1	0	8	19	1
19	RegisterForm	Forms/Auth	11	1	0	4	15	3
20	StudentDashboard	Forms/Student	6	1	0	6	17	10
21	BrowseSocieties	Forms/Student	11	1	0	4	16	4
22	MyMemberships	Forms/Student	4	1	0	3	10	1
23	BrowseEvents	Forms/Student	11	1	0	3	16	4
24	MyTickets	Forms/Student	4	1	0	3	10	1
25	SocietyDashboard	Forms/Society	12	1	0	7	23	21
26	ManageMembers	Forms/Society	15	1	0	3	19	9
27	ManageEvents	Forms/Society	14	1	0	4	19	8
28	CreateEventDialog	Forms/Society	3	1	0	1	5	0
29	ManageTasks	Forms/Society	14	1	0	4	19	8
30	AssignTaskDialog	Forms/Society	3	1	0	2	7	0
31	SocietyReports	Forms/Society	9	1	0	4	14	6
32	AdminDashboard	Forms/Admin	8	1	0	9	21	15
33	ManageUsers	Forms/Admin	14	1	0	2	18	4
34	ManageSocieties	Forms/Admin	19	1	0	3	21	9
35	CreateSocietyDialog	Forms/Admin	3	1	0	2	7	0
36	ApproveEvents	Forms/Admin	12	1	0	2	16	4
37	AdminReports	Forms/Admin	6	1	0	4	13	6

4.2 Interpretation Thresholds

Metric	Low (Good)	Moderate	High (Risky)
WMC	≤ 10	11–20	> 20
DIT	0–1	2–3	> 3
NOC	0	1–3	> 3
CBO	≤ 3	4–7	> 7
RFC	≤ 15	16–25	> 25
LCOM	0	1–10	> 10

4.3 CK Analysis Questions

Q1: What is the maximum depth of inheritance?

The maximum Depth of Inheritance Tree (DIT) is **1**. All 20 Form classes inherit from *System.Windows.Forms.Form* (DIT = 1), while the remaining 17 classes (DAL, Models, Helpers, Config, Program) have DIT = 0, meaning they do not extend any custom class. This flat hierarchy is a positive indicator — it minimises the fragile base-class problem and keeps the system easy to understand.

Q2: Which class has the highest/lowest WMC?

The **highest WMC is ManageSocieties at 19**. It contains 8 methods handling 4 CRUD operations (Create, Approve, Suspend, Delete) plus loading and navigation, each with error-handling paths. The **lowest WMC is 0**, shared by the 6 model classes (User, Society, Event, Membership, TaskItem, Announcement) which are pure data containers with auto-properties and no methods. The next lowest are Session and Program (WMC = 1 each).

Q3: Which class has the greatest number of children?

NOC = 0 for every class. No class in the project is subclassed by another project class. The system uses composition and form-navigation rather than inheritance hierarchies, resulting in a completely flat NOC landscape. This is architecturally sound for a WinForms CRUD application.

Q4: Which class is the most complex?

ManageSocieties is the most complex class with WMC = 19 and RFC = 21. Both metrics are in the moderate-to-high range. Its 8 methods collectively produce 19 decision paths and can potentially invoke 21 distinct methods (including calls to SocietyDAL, UserDAL, MessageBox, and internal helpers). Close runner-ups are ManageMembers (WMC = 15, RFC = 19) and ManageEvents / ManageTasks (WMC = 14, RFC = 19 each).

Q5: Which class has the most coupling?

Session has the highest CBO at 12, but this is *passive* coupling — it is referenced by 12 other classes as a static state holder, rather than actively depending on them. Among classes with *active* outgoing dependencies, **AdminDashboard** has the highest CBO at 9, referencing 9 external classes (3 DAL classes, 4 Form classes, Session, and MessageBox). This high coupling makes AdminDashboard the most change-sensitive class in the system.

Q6: Which class is the least cohesive?

SocietyDashboard has the highest LCOM at 21, making it the least cohesive class. It contains 8 methods, most of which are independent navigation button handlers that share no instance variables with each other.

This is characteristic of a “navigation hub” form where each button click opens a different child form. While the high LCOM is expected for this pattern, it suggests the class could benefit from a command/mediator pattern to decouple navigation responsibilities. The runner-up is AdminDashboard (LCOM = 15), which follows the same hub pattern.

5. Task 5 — Fault Injection & Reliability Analysis

For each module, 5 specific faults were injected. Using the Poisson distribution with $\lambda = 5/(CC+1)$ and confidence threshold E=1, the probability $P(x <= 1) = e^{-\lambda} \times (1 + \lambda)$ was calculated.

5.1 Reliability Summary (All Modules)

#	Module	Faults	CC (WMC)	λ	P(x≤1)	Reliability %	Rank
1	ManageSocieties	5	19	0.2500	0.9735	97.35%	1
2	ManageMembers	5	15	0.3125	0.9603	96.03%	2
3	ManageEvents	5	14	0.3333	0.9553	95.53%	3
4	ManageTasks	5	14	0.3333	0.9553	95.53%	3
5	ManageUsers	5	14	0.3333	0.9553	95.53%	3
6	EventDAL	5	12	0.3846	0.9430	94.30%	6
7	SocietyDashboard	5	12	0.3846	0.9430	94.30%	6
8	ApproveEvents	5	12	0.3846	0.9430	94.30%	6
9	LoginForm	5	11	0.4167	0.9338	93.38%	9
10	RegisterForm	5	11	0.4167	0.9338	93.38%	9
11	BrowseSocieties	5	11	0.4167	0.9338	93.38%	9
12	BrowseEvents	5	11	0.4167	0.9338	93.38%	9
13	UserDAL	5	10	0.4545	0.9228	92.28%	13
14	SocietyDAL	5	9	0.5000	0.9098	90.98%	14
15	MembershipDAL	5	9	0.5000	0.9098	90.98%	14
16	SocietyReports	5	9	0.5000	0.9098	90.98%	14
17	AdminDashboard	5	8	0.5556	0.8926	89.26%	17
18	EnvConfig	5	6	0.7143	0.8394	83.94%	18
19	StudentDashboard	5	6	0.7143	0.8394	83.94%	18
20	AdminReports	5	6	0.7143	0.8394	83.94%	18
21	DBHelper	5	4	1.0000	0.7358	73.58%	21
22	MyMemberships	5	4	1.0000	0.7358	73.58%	21
23	MyTickets	5	4	1.0000	0.7358	73.58%	21
24	TaskDAL	5	3	1.2500	0.6446	64.46%	24
25	CreateEventDialog	5	3	1.2500	0.6446	64.46%	24
26	AssignTaskDialog	5	3	1.2500	0.6446	64.46%	24
27	CreateSocietyDialog	5	3	1.2500	0.6446	64.46%	24
28	AnnouncementDAL	5	2	1.6667	0.5037	50.37%	28
29	PasswordHasher	5	2	1.6667	0.5037	50.37%	28
30	Session	5	1	2.5000	0.2873	28.73%	30
31	Program	5	1	2.5000	0.2873	28.73%	30

5.2 Representative Fault Injection Details

LoginForm (CC=11, Reliability=93.38%)

#	Fault Type	Description	Method
1	Off-by-one	Changed SelectedRows.Count == 0 guard to < 0	BtnLogin_Click
2	Wrong condition	Changed user == null || !Verify(...) to user != null || ...	BtnLogin_Click
3	Null reference	Removed empty email/password check	BtnLogin_Click
4	Wrong logic	Set Session.Role = Admin always	BtnLogin_Click
5	Missing validation	Removed password empty-check	BtnLogin_Click

ManageSocieties (CC=19, Reliability=97.35%)

#	Fault Type	Description	Method
1	Off-by-one	Changed SelectedRows.Count == 0 to < 0	BtnDelete_Click
2	Wrong condition	Called Suspend() instead of Approve()	BtnApprove_Click
3	Null reference	Removed row selection check	BtnSuspend_Click
4	Wrong SQL	Passed UserID instead of SocietyID	BtnApprove_Click
5	Missing validation	Removed confirmation dialog	BtnDelete_Click

DBHelper (CC=4, Reliability=73.58%)

#	Fault Type	Description	Method
1	Off-by-one	Called connection.Open() twice	GetConnection
2	Wrong condition	Used SqlCommand(query, null)	ExecuteNonQuery
3	Null reference	Removed using on connection	ExecuteReader
4	Wrong SQL	Forgot Parameters.AddRange()	ExecuteNonQuery
5	Missing validation	Removed null check on ConnectionString	GetConnection

Session (CC=1, Reliability=28.73%)

#	Fault Type	Description	Method
1	Off-by-one	Set UserID = –1 instead of 0	Clear
2	Wrong condition	Left Role unchanged	Clear
3	Null reference	Set FullName = null instead of empty	Clear
4	Wrong logic	Only cleared UserID	Clear
5	Missing validation	Did not set SocietyID = null	Clear

BrowseSocieties (CC=11, Reliability=93.38%)

#	Fault Type	Description	Method
1	Off-by-one	Changed Count == 0 to <= 0	BtnApply_Click
2	Wrong condition	Negated HasApplied result	BtnApply_Click
3	Null reference	Removed cell null check	BtnApply_Click
4	Wrong SQL	Swapped UserID/SocietyID params	BtnApply_Click
5	Missing validation	Removed selection guard	BtnApply_Click

5.3 Worked Example — LoginForm

Module: LoginForm | **Faults** = 5 | **CC (WMC)** = 11
$\lambda = 5 / (11 + 1) = 5/12 = 0.4167$
$P(x=0) = e^{-0.4167} = 0.6592$
$P(x=1) = 0.4167 \times 0.6592 = 0.2746$
$P(x \leq 1) = 0.6592 + 0.2746 =$ **0.9338 = 93.38%**

5.4 Reliability Conclusions

Most Reliable: ManageSocieties (97.35%) — With the highest cyclomatic complexity (CC=19), this module has the smallest λ value (0.25). Higher complexity paradoxically yields better reliability scores because the formula distributes faults across more paths, reducing the per-path fault density.

Least Reliable: Session / Program (28.73%) — These minimal modules (CC=1) concentrate all 5 faults into a single execution path, producing $\lambda=2.5$. The Poisson probability of at most 1 fault occurring is only 28.73%, indicating extreme vulnerability to any injected fault.

6. Task 6 — KLM Usability Evaluation

KLM Operator Reference: K (Keystroke) = 280 ms, M (Mental preparation) = 1,350 ms, P (Pointing/mouse move) = 1,100 ms, H (Hand switch keyboard↔mouse) = 400 ms.

6.1 KLM Task Summary (All Screens)

#	Screen	Task	K	M	P	H	Total (ms)	Total (s)
1	ManageEvents	Create event	78	6	3	2	34,040	34.04
2	ManageTasks	Assign task	63	6	4	2	30,940	30.94
3	RegisterForm	Register	54	6	4	2	28,420	28.42
4	LoginForm	Log in	29	4	2	2	16,520	16.52
5	ManageUsers	Search+delete	10	5	5	2	15,850	15.85
6	BrowseSocieties	Apply	0	2	2	0	4,900	4.90
7	BrowseEvents	Register	0	2	2	0	4,900	4.90
8	ManageMembers	Approve	0	2	2	0	4,900	4.90
9	ManageSocieties	Approve	0	2	2	0	4,900	4.90
10	ApproveEvents	Approve	0	2	2	0	4,900	4.90
11	SocietyReports	View report	2	1	2	0	4,110	4.11
12	AdminReports	View report	2	1	2	0	4,110	4.11
13	MyMemberships	View status	2	1	1	0	3,010	3.01
14	StudentDashboard	Navigate	0	1	1	0	2,450	2.45
15	SocietyDashboard	Navigate	0	1	1	0	2,450	2.45
16	AdminDashboard	Navigate	0	1	1	0	2,450	2.45
17	MyTickets	View passes	0	1	1	0	2,450	2.45

6.2 Step-by-Step: LoginForm (Log in)

#	Action	Operator	Time (ms)
1	Think about credentials	M	1,350
2	Move to Email field	P	1,100
3	Switch to keyboard	H	400
4	Think before typing email	M	1,350
5	Type email (20 chars)	20K	5,600
6	Tab to Password	K	280
7	Think before typing password	M	1,350
8	Type password (8 chars)	8K	2,240
9	Switch to mouse	H	400
10	Think before Login	M	1,350
11	Click Login button	P	1,100
	TOTAL	**M=4 H=2 P=2 K=29**	**16,520**

6.3 Step-by-Step: RegisterForm (Register)

#	Action	Operator	Time (ms)
1	Think about filling form	M	1,350
2	Move to Full Name field	P	1,100
3	Switch to keyboard	H	400
4	Think before typing name	M	1,350
5	Type full name (15 chars)	15K	4,200
6	Tab to Email field	K	280

#	Action	Operator	Time (ms)
7	Think before typing email	M	1,350
8	Type email (20 chars)	20K	5,600
9	Tab to Password	K	280
10	Think before typing password	M	1,350
11	Type password (8 chars)	8K	2,240
12	Tab to Confirm Password	K	280
13	Type confirm password (8 chars)	8K	2,240
14	Switch to mouse	H	400
15	Think before selecting role	M	1,350
16	Move to Role dropdown	P	1,100
17	Select Student option	P	1,100
18	Think before Register	M	1,350
19	Click Register button	P	1,100
	TOTAL	**M=6 H=2 P=4 K=54**	**28,420**

6.4 Step-by-Step: ManageEvents (Create Event)

#	Action	Operator	Time (ms)
1	Think about creating event	M	1,350
2	Click Create button	P	1,100
3	Dialog: Move to Title field	P	1,100
4	Switch to keyboard	H	400
5	Think before typing title	M	1,350
6	Type title (20 chars)	20K	5,600
7	Tab to Description field	K	280
8	Think before description	M	1,350
9	Type description (30 chars)	30K	8,400
10	Tab to Date picker	K	280
11	Think before date	M	1,350
12	Type date (10 chars)	10K	2,800
13	Tab to Venue field	K	280
14	Think before typing venue	M	1,350
15	Type venue (15 chars)	15K	4,200
16	Switch to mouse	H	400
17	Think before Create	M	1,350
18	Click Create button	P	1,100
	TOTAL	**K=78 M=6 P=3 H=2**	**34,040**

6.5 Overall KLM Statistics

Metric	Value
Average task time	9,553 ms (9.55 s)
Median task time	4,900 ms (4.90 s)
Total keyboard time	67,200 ms (240 K)
Total mental time	54,000 ms (40 M)
Total pointing time	46,200 ms (42 P)
Total hand-switch time	4,800 ms (12 H)

6.6 Performance Analysis

Fastest Screens (2,450 ms): Dashboard and navigation screens (StudentDashboard, SocietyDashboard, AdminDashboard, MyTickets) require only a single mental preparation and one pointing action. These are read-only views with minimal interaction.

Slowest Screen (34,040 ms): ManageEvents requires 78 keystrokes across multiple form fields using Tab navigation, 6 mental preparations, and 3 pointing actions. The high KLM time reflects the data-entry-heavy nature of event creation.

Key Insight: Keyboard input (K) dominates total interaction time at 67,200 ms (39.0%), followed by mental preparation (M) at 54,000 ms (31.4%). Optimizing form entry with auto-fill, defaults, and Tab-order improvements would yield the greatest usability gains.

7. Task 7 — COCOMO Model

Selected Model: Basic COCOMO | **Mode:** Semi-detached
Semi-detached mode coefficients: a=3.0, b=1.12, c=2.5, d=0.35

7.1 Per-File Lines of Code

#	File	Physical LOC	Blank	Comments	Code
1	EnvConfig.cs	60	9	16	35
2	DBHelper.cs	61	5	17	39
3	UserDAL.cs	123	14	29	80
4	SocietyDAL.cs	112	11	30	71
5	MembershipDAL.cs	134	17	30	87
6	EventDAL.cs	162	18	39	105
7	TaskDAL.cs	55	6	12	37
8	AnnouncementDAL.cs	38	4	9	25
9	Session.cs	31	2	10	19
10	PasswordHasher.cs	24	1	9	14
11	Program.cs	21	2	4	15
12	User.cs	15	0	3	12
13	Society.cs	15	0	3	12
14	Event.cs	17	0	3	14
15	Membership.cs	15	0	3	12
16	TaskItem.cs	19	0	4	15
17	Announcement.cs	14	0	3	11
18	LoginForm.cs	78	9	9	60
19	LoginForm.Designer.cs	103	14	8	81
20	RegisterForm.cs	95	9	10	76
21	RegisterForm.Designer.cs	145	20	14	111
22	StudentDashboard.cs	51	6	6	39
23	StudentDashboard.Designer.cs	101	14	8	79
24	BrowseSocieties.cs	81	8	9	64
25	BrowseSocieties.Designer.cs	89	12	6	71
26	MyMemberships.cs	35	3	3	29
27	MyMemberships.Designer.cs	70	10	4	56
28	BrowseEvents.cs	78	8	6	64
29	BrowseEvents.Designer.cs	89	12	6	71
30	MyTickets.cs	35	3	3	29
31	MyTickets.Designer.cs	70	10	4	56
32	SocietyDashboard.cs	98	10	7	81
33	SocietyDashboard.Designer.cs	111	15	9	87
34	ManageMembers.cs	94	12	9	73
35	ManageMembers.Designer.cs	100	13	7	80
36	ManageEvents.cs	159	21	13	125
37	ManageEvents.Designer.cs	89	12	6	71
38	ManageTasks.cs	162	21	12	129
39	ManageTasks.Designer.cs	89	12	6	71
40	SocietyReports.cs	58	7	3	48
41	SocietyReports.Designer.cs	131	16	10	105
42	AdminDashboard.cs	66	7	6	53
43	AdminDashboard.Designer.cs	140	18	12	110
44	ManageUsers.cs	88	10	9	69
45	ManageUsers.Designer.cs	97	13	7	77

#	File	Physical LOC	Blank	Comments	Code
46	ManageSocieties.cs	177	20	9	148
47	ManageSocieties.Designer.cs	107	14	8	85
48	ApproveEvents.cs	81	8	9	64
49	ApproveEvents.Designer.cs	89	12	6	71
50	AdminReports.cs	46	5	3	38
51	AdminReports.Designer.cs	140	17	11	112
	TOTAL	**4,158**	**490**	**482**	**3,186**

7.2 LOC Summary

Metric	Value
Total Files	51
Physical LOC	4,158
Blank Lines	490
Comment Lines	482
Code Lines	3,186
KLOC (Physical)	4.158
KLOC (Logical/Code)	3.186

7.3 Model Justification

Semi-detached mode was selected because: (1) the system involves multiple user roles (Admin, Society Head, Student) with distinct interfaces; (2) it integrates with a SQL Server database through a custom DAL layer; (3) the development team has mixed experience levels with the chosen technology stack (C# WinForms + SQL Server); and (4) the project size (~4 KLOC) falls within the semi-detached range (2–50 KLOC).

7.4 COCOMO Calculation

Parameter	Formula	Calculation	Result
KLOC	LOC / 1000	4158 / 1000	4.158
Effort (E)	3.0 × (KLOC)^1.12	3.0 × 4.9279	14.78 PM
Duration (D)	2.5 × (E)^0.35	2.5 × 2.5674	6.42 months
Team Size (P)	E / D	14.78 / 6.42	2.30 ≈ 3

7.5 Estimate vs. Actual

Metric	COCOMO Estimate	Actual
Effort	14.78 Person-Months	~3 Person-Months
Duration	6.42 months	~1 month
Team Size	~3 persons	3 persons

7.6 Analysis

The Basic COCOMO model significantly overestimates both effort (14.78 vs ~3 PM) and duration (6.42 vs ~1 month). This discrepancy is attributable to several factors:

1. AI-Assisted Development: Large portions of the codebase (especially Designer files and DAL boilerplate) were generated with AI assistance, dramatically reducing manual coding time.
2. Template Reuse: WinForms Designer files follow repetitive patterns; once one form was designed, others were cloned and adapted.
3. Model Limitations: Basic COCOMO does not account for modern development tools, code generation, or framework productivity multipliers.
4. Team familiarity grew quickly with the relatively simple architecture (3-tier, single DB, no distributed components).

8. Task 8 — Documentation Ratio

Formula: Documentation Ratio = Total LOC / Comment Lines (lower is better).
A lower ratio indicates more comments per line of code, implying better documentation coverage.

8.1 Per-File Documentation Ratio

#	File	Total LOC	Comments	Ratio	Classification
1	EnvConfig.cs	60	16	3.75	Well-documented
2	Program.cs	21	4	5.25	Average
3	DBHelper.cs	61	17	3.59	Well-documented
4	UserDAL.cs	123	29	4.24	Well-documented
5	SocietyDAL.cs	112	30	3.73	Well-documented
6	MembershipDAL.cs	134	30	4.47	Well-documented
7	EventDAL.cs	162	39	4.15	Well-documented
8	TaskDAL.cs	55	12	4.58	Well-documented
9	AnnouncementDAL.cs	38	9	4.22	Well-documented
10	Session.cs	31	10	3.10	Well-documented
11	PasswordHasher.cs	24	9	2.67	Well-documented
12	User.cs	15	3	5.00	Average
13	Society.cs	15	3	5.00	Average
14	Event.cs	17	3	5.67	Average
15	Membership.cs	15	3	5.00	Average
16	TaskItem.cs	19	4	4.75	Well-documented
17	Announcement.cs	14	3	4.67	Well-documented
18	LoginForm.cs	78	9	8.67	Average
19	RegisterForm.cs	95	10	9.50	Average
20	StudentDashboard.cs	51	6	8.50	Average
21	BrowseSocieties.cs	81	9	9.00	Average
22	MyMemberships.cs	35	3	11.67	Under-documented
23	BrowseEvents.cs	78	6	13.00	Under-documented
24	MyTickets.cs	35	3	11.67	Under-documented
25	SocietyDashboard.cs	98	7	14.00	Under-documented
26	ManageMembers.cs	94	9	10.44	Under-documented
27	ManageEvents.cs	159	13	12.23	Under-documented
28	ManageTasks.cs	162	12	13.50	Under-documented
29	SocietyReports.cs	58	3	19.33	Poorly-documented
30	AdminDashboard.cs	66	6	11.00	Under-documented
31	ManageUsers.cs	88	9	9.78	Average
32	ManageSocieties.cs	177	9	19.67	Poorly-documented
33	ApproveEvents.cs	81	9	9.00	Average
34	AdminReports.cs	46	3	15.33	Poorly-documented
35	LoginForm.Designer.cs	103	8	12.88	Under-documented
36	RegisterForm.Designer.cs	145	14	10.36	Under-documented
37	StudentDashboard.Designer.cs	101	8	12.63	Under-documented
38	BrowseSocieties.Designer.cs	89	6	14.83	Under-documented
39	MyMemberships.Designer.cs	70	4	17.50	Poorly-documented
40	BrowseEvents.Designer.cs	89	6	14.83	Under-documented
41	MyTickets.Designer.cs	70	4	17.50	Poorly-documented
42	SocietyDashboard.Designer.cs	111	9	12.33	Under-documented
43	ManageMembers.Designer.cs	100	7	14.29	Under-documented
44	ManageEvents.Designer.cs	89	6	14.83	Under-documented
45	ManageTasks.Designer.cs	89	6	14.83	Under-documented

#	File	Total LOC	Comments	Ratio	Classification
46	SocietyReports.Designer.cs	131	10	13.10	Under-documented
47	AdminDashboard.Designer.cs	140	12	11.67	Under-documented
48	ManageUsers.Designer.cs	97	7	13.86	Under-documented
49	ManageSocieties.Designer.cs	107	8	13.38	Under-documented
50	ApproveEvents.Designer.cs	89	6	14.83	Under-documented
51	AdminReports.Designer.cs	140	11	12.73	Under-documented

8.2 Project-Wide Summary

Metric	Value
Total Physical LOC	4,158
Total Comment Lines	482
Project Ratio	8.63
Classification	Average

8.3 Classification Distribution

Classification	Criteria	Count	%
Well-documented	< 5	12	23.5%
Average	5 – 10	11	21.6%
Under-documented	10 – 15	23	45.1%
Poorly-documented	> 15	5	9.8%

8.4 Best & Worst Documented Files

Category	File	Ratio
Best	PasswordHasher.cs	2.67
2nd best	Session.cs	3.10
3rd best	DBHelper.cs	3.59
Worst	ManageSocieties.cs	19.67
2nd worst	SocietyReports.cs	19.33

8.5 Documentation by Module Category

Category	Files	Avg Ratio
Config + Entry	2	4.50
DAL (Data Access)	7	4.14
Helpers	2	2.89
Models	6	5.02
Forms (code-behind)	17	12.13
Forms (Designer)	17	13.89

8.6 Vibe-Coding Documentation Analysis

1. Infrastructure code receives the most documentation — DAL and helper files (ratios 2.67–4.58) contain XML documentation comments on every public method and class, reflecting deliberate effort to

document the data-access API.

2. UI code-behind has declining documentation quality — Form logic files average a ratio of 12.13. Event handlers and UI wiring are often self-explanatory but lack summary comments explaining complex workflows.

3. Designer files follow a fixed template — Auto-generated Designer.cs files have ratios between 10–17.5. Their comments are boilerplate (#region, component declarations) and do not reflect intentional documentation effort.

4. Model classes have minimal but adequate documentation — With an average ratio of 5.02, model files contain brief XML summary tags on properties, which is standard practice for simple POCO/DTO classes.

5. Project-wide ratio of 8.63 (Average) — This is acceptable for a vibe-coded university project. The strong documentation in infrastructure layers compensates for lighter coverage in UI code, resulting in a balanced overall ratio.